

Projet de Calcul Stochastique

Réalisé par Carine MAJCHRZAK (ARM) et Davy TORRO (ARB)

Introduction

Fondé le 7 mai 1946, Sony dont le nom sera donné en 1955 car il signifie un esprit jeune "Son", innovant et libre avec comme formule "fais ce qui n'a jamais été fait auparavant". Objectif accompli puisque c'est une société qui deviendra un des plus grands conglomérats économiques mondiaux c'est-à-dire une société qui possède des activités dans différents secteurs tel que le cinéma, l'industrie électronique, les jeux-vidéos, la musique, la photographie, les téléphones et d'autres industries audiovisuelles encore. Le groupe renommé "Sony Group" en 2021 possède plus d'une centaine de sociétés à travers le monde grâce à sa prise de risque au tout départ de l'évolution technologique que nous connaissons désormais. En effet, elle a pris le risque d'acheter un brevet de la CFT (Compagnie Française de la Télévision) afin de lancer le "Trinitron" qui sera lancé et commercialisé en 1968 en France, ancêtre des télévisions aux écrans à cristaux liquides (LCD). Elle a également innové son temps avec le "Walkman" en 1979 que sont les baladeurs à cassettes, et le "Compact Disc" (CD) en coopération avec Philips en 1982 qui révolutionneront leur milieu. C'est aussi Sony qui développera ces derniers et créera le premier caméscope en 1983, lance le "Digital Versatile Disc" (DVD) en 1995 et le disque Blu-Ray en 2006. Et puis en marge de tout cela, ils vont se placer sur la création de la Playstation en 1994 dans une industrie qui n'est pas encore totalement démocratisé et par ailleurs moquer à l'époque. Nous tenons donc à réaliser ce projet sur cette société qui nous accompagne depuis près d'un siècle et qui ne cesse de progresser avec son temps. De plus, le monde moderne est très assimilé à la technologie, et d'autant plus depuis la révolution numérique que constitue internet, le marché s'est donc ouvert à d'autres acteurs ainsi qu'à une demande plus amplement élargie et diversifiée.

Objectif

Ce projet consistera à effectuer une calibration de la volatilité implicite qui sera dérivée du prix de l'option. Cela nous permettra de modéliser la surface de volatilité implicite, essentiel pour une analyse complète des anticipations du marché concernant la valeur de l'actif sous-jacent. Pour réaliser cette analyse, nous utiliserons le modèle de Black-Scholes inversée et le modèle de Heston.

Présentation de nos données

L'actif sous-jacent

L'actif sous-jacent représenté dans notre modèle sera SONY libellé en USD et coté au New York Stock Exchange. Nous prendrons, dans le cadre de notre analyse, les options d'achats

(calls) européens sur différentes maturités de l'option.

Base de donnée

Le tableau ci-dessous affiche notre base de données correspondant à la première échéance, nous ferons de même pour six échéances dispersées dans un horizon temporel défini. Pour l'optimisation de notre processus de programmation, nous avons décidé de supprimer les pourcentages de la volatilité implicite et de représenter la valeur sous sa forme "absolue".

In [11]: data_call_t1

Out[11]:

	Nom du contrat	Date de la dernière transaction	Prix d'exercice	Dernier prix	Offre	Vente	Variation	% de variation	Vol
0	SONY231222C00086000	2023-12-12 10:43AM EST	86	4.30	5.70	6.00	0.00	-	
1	SONY231222C00087000	2023-12-12 10:18AM EST	87	3.30	3.20	5.00	0.00	-	
2	SONY231222C00088000	2023-12-11 2:05PM EST	88	2.15	3.80	4.00	0.00	-	
3	SONY231222C00089000	2023-12-13 2:51PM EST	89	2.60	2.80	3.10	0.00	-	
4	SONY231222C00090000	2023-12-15 9:48AM EST	90	3.40	1.95	2.20	0.00	-	
5	SONY231222C00091000	2023-12-14 12:28PM EST	91	2.00	1.30	1.45	0.00	-	
6	SONY231222C00092000	2023-12-18 9:57AM EST	92	0.83	0.75	0.90	-0.66	-0.443	
7	SONY231222C00093000	2023-12-18 9:46AM EST	93	0.37	0.35	0.45	-0.56	-0.6022	
8	SONY231222C00094000	2023-12-18 9:35AM EST	94	0.18	0.15	0.25	-0.37	-0.6727	
9	SONY231222C00095000	2023-12-18 9:30AM EST	95	0.20	0.05	0.15	-0.15	-0.4286	
10	SONY231222C00096000	2023-12-15 1:40PM EST	96	0.10	0.00	0.10	-0.10	-0.5	
11	SONY231222C00100000	2023-11-14 1:33PM EST	100	0.20	0.00	0.10	0.00	-	

Représentation du modèle Black-Scholes inversée

Installation des packages

```
In [13]: import pandas as pd
import numpy as np
import matplotlib as plt
import QuantLib as ql
from mpl_toolkits.mplot3d import Axes3D
import matplotlib.pyplot as plt
from matplotlib import cm
from scipy.stats import norm
from scipy.optimize import brentq
from scipy.optimize import newton
import scipy.optimize as optimize
from scipy.optimize import minimize
```

Lecture des données et affectation des variables

```
In [8]: fichier_excel = 'C:/Users/Davy/Desktop/Sony.xlsx'

# Lire le fichier Excel

data_call_t1 = pd.read_excel(fichier_excel, sheet_name='22 Dec')
data_call_t2 = pd.read_excel(fichier_excel, sheet_name='29 Dec')
data_call_t3 = pd.read_excel(fichier_excel, sheet_name='5 Janv')
data_call_t4 = pd.read_excel(fichier_excel, sheet_name='12 Janv')
data_call_t5 = pd.read_excel(fichier_excel, sheet_name='19 Janv')
data_call_t6 = pd.read_excel(fichier_excel, sheet_name='26 Janv')

#Volatilité implicite calculée à partir de la formule inversée du modèle Black-Scholes

volatilite_impl_t1 = data_call_t1['Volatilité implicite']
volatilite_impl_t2 = data_call_t2['Volatilité implicite']
volatilite_impl_t3 = data_call_t3['Volatilité implicite']
volatilite_impl_t4 = data_call_t4['Volatilité implicite']
volatilite_impl_t5 = data_call_t5['Volatilité implicite']
volatilite_impl_t6 = data_call_t6['Volatilité implicite']

#Les différents Strikes exercés sur Le marché
K1 = data_call_t1['Prix d'exercice']
K2 = data_call_t2['Prix d'exercice']
K3 = data_call_t3['Prix d'exercice']
K4 = data_call_t4['Prix d'exercice']
K5 = data_call_t5['Prix d'exercice']
K6 = data_call_t6['Prix d'exercice']

#Valeur de l'actif sous-jacent
S = 91.64

#Taux d'intérêt sans risque repris à partir du taux du trésor américain à 1 mois
r = 5.378/100

#Prix des options quotés sur Le marché
option_prices_t1 = data_call_t1['Dernier prix']
option_prices_t2 = data_call_t2['Dernier prix']
option_prices_t3 = data_call_t3['Dernier prix']
option_prices_t4 = data_call_t4['Dernier prix']
option_prices_t5 = data_call_t5['Dernier prix']
option_prices_t6 = data_call_t6['Dernier prix']
```

Calibration de la volatilité implicite

```
In [24]: # Black-Scholes Formula
def black_scholes_call(S, K, T, r, sigma):
    d1 = (np.log(S / K) + (r + 0.5 * sigma ** 2) * T) / (sigma * np.sqrt(T))
    d2 = d1 - sigma * np.sqrt(T)
    return S * norm.cdf(d1) - K * np.exp(-r * T) * norm.cdf(d2)

# Calculating Implied Volatility
def implied_volatility_call(price, S, K, T, r):
    def f(sigma): # Function to find the root
        return black_scholes_call(S, K, T, r, sigma) - price

    def vega(S, K, T, r, sigma): # Vega for Newton-Raphson method
        d1 = (np.log(S / K) + (r + 0.5 * sigma ** 2) * T) / (sigma * np.sqrt(T))
        return S * norm.pdf(d1) * np.sqrt(T)

    return newton(f, 0.2, fprime=lambda sigma: vega(S, K, T, r, sigma)) # Initial

volatilite_impl_t1
```

```
Out[24]: 0    0.4990
1    0.4351
2    0.3691
3    0.3340
4    0.2857
5    0.2588
6    0.2500
7    0.2285
8    0.2373
9    0.2529
10   0.2754
11   0.4482
Name: Volatilité implicite, dtype: float64
```

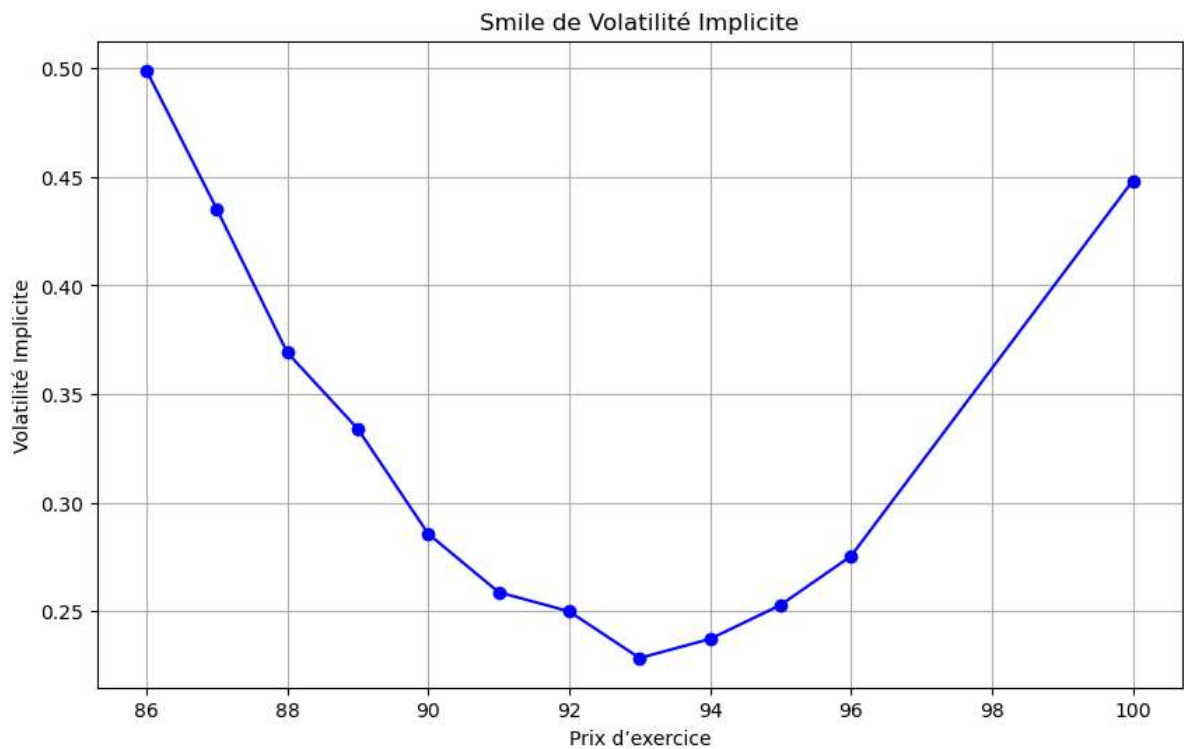
Smile de volatilité implicite

Le smile de volatilité nous permet d'évaluer les différents niveaux de volatilités implicite sur différents prix d'exercice pour une échéance donnée. Ce smile contredit la théorie du modèle Black-Scholes car cette volatilité varie en fonction du prix d'exercice. Il fournit également une multitude d'informations sur la perception des risques du marché selon la pente et la convexité de la courbe.

```
In [27]: # Trier les données en fonction du prix d'exercice
K1, volatilite_impl_t1 = zip(*sorted(zip(K1, volatilite_impl_t1)))

# Création du graphique
plt.figure(figsize=(10, 6))
plt.plot(K1, volatilite_impl_t1, marker='o', linestyle='-', color='b')
plt.title('Smile de Volatilité Implicite')
plt.xlabel('Prix d'exercice')
plt.ylabel('Volatilité Implicite')
plt.grid(True)

# Afficher le graphique
plt.show()
```



Le graphique montre des niveaux de volatilité implicite plus élevés aux extrémités des prix d'exercice et plus faibles au centre. Ceci est caractéristique d'un marché où les traders anticipent une plus grande incertitude ou des mouvements de prix conséquent loin du prix actuel de l'actif sous-jacent. En effet, on constate que la volatilité implicite est plus élevée pour les options "out of the money" et plus basse pour les options "at the money". On peut en déduire que ce smile est bien plus représentatif du marché que le modèle Black-Scholes initial car il reflète les événements et émotions des acteurs notamment sur des potentiels annonces économiques importantes, des résultats d'entreprise, ou d'autres événements susceptibles d'affecter l'actif sous-jacent. Ainsi un trader qui pense que le marché va rester relativement stable pourrait mettre en place un butterfly spread, vendant deux options at-the-money où la volatilité implicite est la plus basse et achetant des options out-of-the-money et in-the-money où la volatilité implicite est plus élevée. S'il croit que le marché va se déplacer significativement, il pourrait acheter des options out-of-the-money qui sont actuellement évaluées avec une volatilité implicite plus élevée. Si le mouvement du marché est suffisamment important, le gain sur ces options pourrait surpasser la prime plus élevée payée due à la volatilité implicite.

Surface de volatilité implicite

La surface de volatilité implicite représente autour de trois axes la structure à terme de la volatilité implicite. Contrairement au smile, celle-ci sera représentée à travers différentes échéances des options. Par ailleurs, elle permet d'obtenir une modélisation plus complète et précise des attentes du marché sur un horizon temporel plus élargi. Ainsi il sera nettement plus aisé d'analyser les anomalies, calibrer un modèle de pricing et d'être réactif aux mouvements du marché.

```
In [72]: # Configuration de la date de calcul
         calculation_date = ql.Date(18, 12, 2023)
         ql.Settings.instance().evaluationDate = calculation_date
```

```

# Dates d'expiration des options
expiration_dates = [
    ql.Date(22, 12, 2023), ql.Date(29, 12, 2023), ql.Date(5, 1, 2024),
    ql.Date(12, 1, 2024), ql.Date(19, 1, 2024), ql.Date(26, 1, 2024)
]

# Maturités calculées en années par rapport à la date de calcul
maturities = [ql.Actual365Fixed().yearFraction(calculation_date, date) for date in expiration_dates]

# Préparation des données pour la surface de volatilité
strikes_flattened = np.concatenate([df['Prix d'exercice'].values for df in [data_calls, data_puts]])
volatilities_flattened = np.concatenate([df['Volatilité implicite'].values for df in [data_calls, data_puts]])
prix_flattened = np.concatenate([df['Dernier prix'].values for df in [data_calls, data_puts]])

# Construction de la surface de volatilité
calendar = ql.France()
day_count = ql.Actual365Fixed()
# Construisez la matrice de volatilité implicite
implied_vols = ql.Matrix(len(strikes_flattened) // len(expiration_dates), len(expiration_dates))
for i in range(implied_vols.rows()):
    for j in range(implied_vols.columns()):
        idx = i * len(expiration_dates) + j
        implied_vols[i][j] = float(volatilities_flattened[idx])

# Création de la surface de volatilité noire de QuantLib
black_var_surface = ql.BlackVarianceSurface(
    calculation_date, calendar, expiration_dates,
    strikes_for_surface, # Utilisez strikes_for_surface ici
    implied_vols, day_count
)

# Supposons que strikes_flattened contient toutes les valeurs du strike
strike_min = min(strikes_flattened)
strike_max = max(strikes_flattened)

# Affichage de la surface de volatilité en 3D
strike_grid = np.linspace(strike_min, strike_max, 50)

# Utilisez les maturités min et max calculées à partir des données d'expiration
maturity_grid = np.linspace(min(maturities), max(maturities), 50)

# Création des grilles pour la visualisation
X, Y = np.meshgrid(strike_grid, maturity_grid)
Z = np.zeros_like(X)

# Utilisez extrapolate=True pour permettre des requêtes en dehors du domaine de la surface
for i in range(X.shape[0]):
    for j in range(X.shape[1]):
        Z[i, j] = black_var_surface.blackVol(Y[i, j], X[i, j], True)

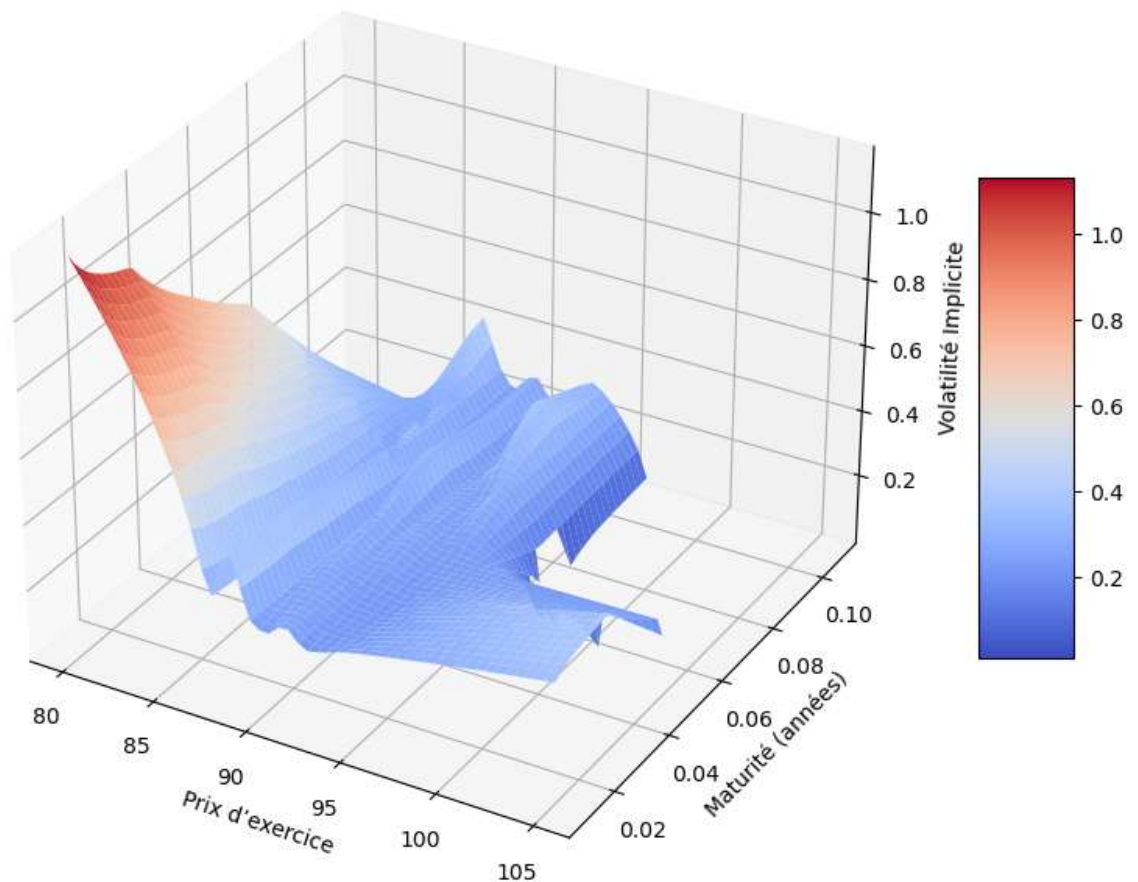
# Tracer la surface de volatilité
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')
surf = ax.plot_surface(X, Y, Z, cmap=cm.coolwarm, linewidth=0)
ax.set_xlabel('Prix d'exercice')
ax.set_ylabel('Maturité (années)')
ax.set_zlabel('Volatilité Implicite')
plt.title('Surface de Volatilité Implicite du Modèle Black-Sholes inversée')

fig.colorbar(surf, shrink=0.5, aspect=5)

plt.show()

```

Surface de Volatilité Implicite du Modèle Black-Sholes inversée



Introduction des différentes maturités

```
In [32]: # Paramètres de base pour QuantLib
calculacion_date = ql.Date.todayDate()
ql.Settings.instance().evaluationDate = calculacion_date

# Création des objets QuantLib
spot_handle = ql.QuoteHandle(ql.SimpleQuote(S))
flat_ts = ql.YieldTermStructureHandle(ql.FlatForward(calculacion_date, ql.QuoteHandle(

# Définir les dates d'expiration et les strikes correspondants
# Vous devrez adapter cette partie en fonction de la structure de vos données
expiration_dates = [ql.Date(22, 12, 2023), ql.Date(29, 12, 2023), ql.Date(5, 1, 2024)]
] # Ajouter toutes les dates d'expiration
strikes = [K1, K2, K3, K4, K5, K6] # Ajouter tous les K
vols = [volatilite_impl_t1, volatilite_impl_t2, volatilite_impl_t3, volatilite_impl_t4]
```

```
In [75]: maturities
```

```
Out[75]: [0.010958904109589041,
0.030136986301369864,
0.049315068493150684,
0.0684931506849315,
0.08767123287671233,
0.10684931506849316]
```

Calibration à partir du modèle de Heston

Le modèle de Black-Scholes émet l'hypothèse que la volatilité du sous-jacent est constante et connue, cependant le modèle de Heston implémente de son côté une volatilité suivant un processus stochastique. C'est un facteur essentiel qui lui permet de posséder une vision plus réaliste du marché de par son caractère aléatoire, ce qui permet une meilleure prise en compte des fluctuations significatives. Par ailleurs, dans une analyse à dimension temporelle plus éloignée, il convient de réaliser ce modèle tandis que le modèle de Black-Scholes de par sa volatilité constante se limite très rapidement à un horizon restreint.

Optimisation des paramètres

Le modèle présente des paramètres essentiels qui auront une part importante dans l'explication de la volatilité du sous-jacent. La calibration consistera à optimiser la valeur des paramètres afin que les prix des options du modèle minimise la marge d'erreur avec les prix réellement observés sur le marché. - Kappa désigne la vitesse de retour à la moyenne à savoir son niveau de long terme (theta). Si le paramètre possède une valeur élevée, alors il s'ajustera rapidement vers theta. - Theta représente ce niveau de long terme de la variance. - Sigma mesure la volatilité de la volatilité ou l'intensité des variations du sous-jacent. - Rho explicite la corrélation entre le rendement de l'actif sous-jacent et sa volatilité. - V0 désigne la variance initiale qui apparaît au début du processus de calibration.

```
In [35]: # Fonction pour créer la structure des données de marché (strikes et volatilités)
def create_market_data_structure(expiration_dates, strikes, vols):
    market_data = []
    for i in range(len(expiration_dates)):
        for j in range(len(strikes[i])):
            market_data.append((expiration_dates[i], strikes[i][j], vols[i][j]))
    return market_data

market_data = create_market_data_structure(expiration_dates, strikes, vols)

def clean_volatility(vol_series):
    return vol_series.astype(float)

def clean_and_convert_volatility(vol_tuple):
    cleaned_vol = []
    for vol in vol_tuple:
        if isinstance(vol, str):
            # Supprimer les pourcentages et convertir en flottant
            vol = vol.replace('%', '')
            vol = float(vol) / 100
        cleaned_vol.append(vol)
    return np.array(cleaned_vol)

# Nettoyage et conversion des données de volatilité
vols_cleaned = [clean_and_convert_volatility(vol) for vol in vols]

# Calcul de la volatilité implicite moyenne
mean_volatility = np.mean([np.mean(vol) for vol in vols_cleaned])

# Définir la structure de volatilité constante
flat_vol_ts = ql.BlackVolTermStructureHandle(ql.BlackConstantVol(calculation_date,

# Création du processus Black-Scholes pour la volatilité implicite
bs_process = ql.BlackScholesProcess(spot_handle, flat_ts, flat_vol_ts)

def clean_and_convert_volatility1(vol):
```

```

    if isinstance(vol, str):
        vol = vol.replace('%', '').strip()
        return float(vol) / 100
    return vol

# Mise à jour de la structure de market_data
market_data_cleaned = []
for date, strike, vol in market_data:
    strike_float = float(strike) # Convertir Le strike en float
    vol_cleaned = clean_and_convert_volatility1(vol)
    market_data_cleaned.append((date, strike_float, vol_cleaned))

# Définition de la fonction de calibration
def calibration_error(params, market_data):
    kappa, theta, sigma, rho, v0 = params
    heston_process = ql.HestonProcess(flat_ts, ql.YieldTermStructureHandle(ql.FlatTermStructure()))
    heston_model = ql.HestonModel(heston_process)
    engine = ql.AnalyticHestonEngine(heston_model)

    total_error = 0.0
    for date, strike, implied_vol in market_data:
        maturity = ql.Period(date - calculation_date, ql.Days)
        payoff = ql.PlainVanillaPayoff(ql.Option.Call, strike)
        exercise = ql.EuropeanExercise(date)
        option = ql.VanillaOption(payoff, exercise)
        option.setPricingEngine(engine)

        try:
            model_iv = option.impliedVolatility(option.NPV(), bs_process, 0.001, 10)
            total_error += (model_iv - implied_vol) ** 2
        except RuntimeError:
            # Gérer l'erreur si la volatilité implicite n'est pas trouvée
            total_error += large_penalty # Ajouter une grande pénalité à l'erreur

    return total_error

# Paramètres initiaux pour la calibration
initial_guess = [0.1, 0.1, 0.1, 0.1, 0.1]
bounds = [(0.001, 10), (0.001, 10), (0.001, 10), (-1, 1), (0.001, 10)]

# Optimisation
optimal_params = optimize.minimize(calibration_error, initial_guess, args=(market_data_cleaned, bounds))

# Affichage des résultats de la calibration
print("Les paramètres calibrés du modèle de Heston sont :")
print("Kappa:", optimal_params.x[0])
print("Theta:", optimal_params.x[1])
print("Sigma:", optimal_params.x[2])
print("Rho:", optimal_params.x[3])
print("V0:", optimal_params.x[4])

```

Les paramètres calibrés du modèle de Heston sont :

Kappa: 0.10000935469569598
 Theta: 0.09936801976343605
 Sigma: 0.0995338453053965
 Rho: 0.09945868648662327
 V0: 0.05279197516179948

Surface de volatilité implicite du modèle

```

In [67]: fichier_excel = 'C:/Users/Davy/Desktop/Sony.xlsx'
        sheets = ['22 Dec', '29 Dec', '5 Janv', '12 Janv', '19 Janv', '26 Janv']
        data_calls = [pd.read_excel(fichier_excel, sheet_name=sheet) for sheet in sheets]

```

```

# Configuration de la date de calcul
calculation_date = ql.Date(18, 12, 2023)
ql.Settings.instance().evaluationDate = calculation_date

# Création des objets QuantLib
spot_handle = ql.QuoteHandle(ql.SimpleQuote(S))
flat_ts = ql.YieldTermStructureHandle(ql.FlatForward(calculation_date, r, ql.Actual
flat_vol_ts = ql.BlackVolTermStructureHandle(ql.BlackConstantVol(calculation_date,

# Paramètres initiaux pour la calibration du modèle de Heston
v0 = 0.1 # Variance initiale
kappa = 0.1 # Taux de retour à la moyenne
theta = 0.1 # Variance à long terme
sigma = 0.1 # Volatilité de la volatilité
rho = -0.1 # Corrélation entre le sous-jacent et la variance

# Création du processus de Heston avec les paramètres initiaux
heston_process = ql.HestonProcess(
    flat_ts, flat_ts, spot_handle, v0, kappa, theta, sigma, rho
)

# Fonction de calibration
def calibration_error(params):
    kappa, theta, sigma, rho, v0 = params
    # Create a new Heston process with the current parameters
    temp_process = ql.HestonProcess(
        flat_ts, flat_ts, spot_handle, v0, kappa, theta, sigma, rho
    )
    temp_model = ql.HestonModel(temp_process)
    engine = ql.AnalyticHestonEngine(temp_model)
    error = 0.0
    for i, data_call in enumerate(data_calls):
        for index, row in data_call.iterrows():
            maturity_date = expiration_dates[i]
            strike = row['Prix d'exercice']
            market_price = row['Dernier prix']
            payoff = ql.PlainVanillaPayoff(ql.Option.Call, strike)
            exercise = ql.EuropeanExercise(maturity_date)
            option = ql.VanillaOption(payoff, exercise)
            option.setPricingEngine(engine)
            model_price = option.NPV()
            error += (model_price - market_price)**2
    return error

# Paramètres initiaux pour la calibration du modèle de Heston
initial_guess = [0.1, 0.1, 0.1, -0.1, 0.1]
bounds = [(0.001, 10), (0.001, 10), (0.001, 10), (-1, 1), (0.001, 10)]

# Calibrer le modèle de Heston
optimal_params = minimize(calibration_error, initial_guess, bounds=bounds, method='

# Utiliser les paramètres calibrés pour créer un nouveau processus Heston
kappa, theta, sigma, rho, v0 = optimal_params.x
calibrated_heston_process = ql.HestonProcess(
    flat_ts, flat_ts, spot_handle, v0, kappa, theta, sigma, rho
)

# Création du modèle de Heston calibré
calibrated_heston_model = ql.HestonModel(calibrated_heston_process)

# Création de l'engine de pricing pour le modèle de Heston calibré
calibrated_heston_engine = ql.AnalyticHestonEngine(calibrated_heston_model)

```

```

# Générer la surface de volatilité implicite
num_strikes = 50
num_maturities = 50
strike_grid = np.linspace(min(strikes_flattened), max(strikes_flattened), num_strikes)
maturity_grid = np.linspace(min(maturities), max(maturities), num_maturities)
implied_vol_surface = np.zeros((num_strikes, num_maturities))

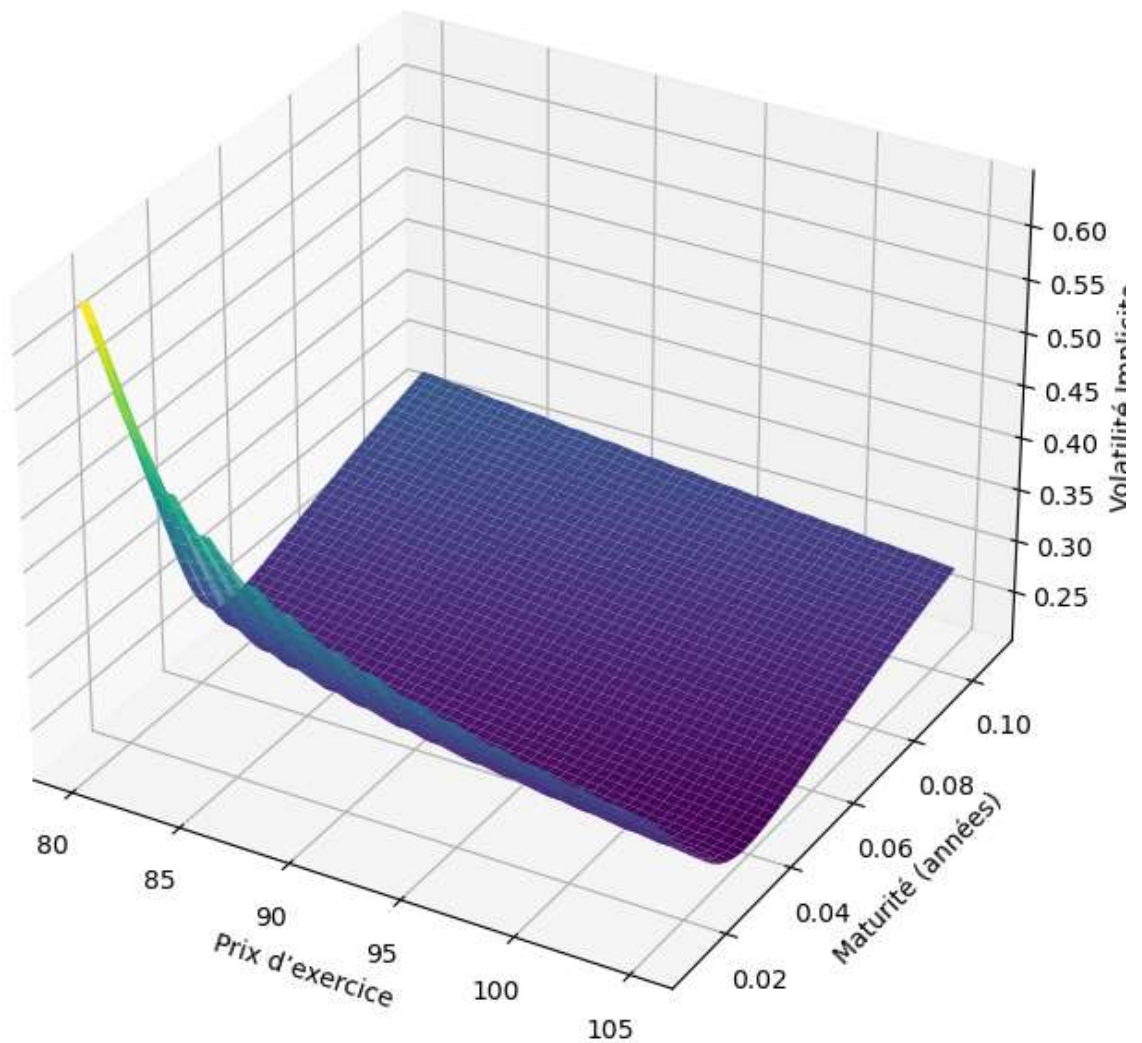
# Calculer la volatilité implicite pour chaque paire de strike et maturité
for i, strike in enumerate(strike_grid):
    for j, t in enumerate(maturity_grid):
        expiry_date = calculation_date + ql.Period(int(t * 365), ql.Days)
        payoff = ql.PlainVanillaPayoff(ql.Option.Call, strike)
        exercise = ql.EuropeanExercise(expiry_date)
        option = ql.VanillaOption(payoff, exercise)
        option.setPricingEngine(calibrated_heston_engine)
        model_price = option.NPV()

        # Utiliser le processus de Black-Scholes pour calculer la volatilité implicite
        bs_process = ql.BlackScholesMertonProcess(
            spot_handle,
            ql.YieldTermStructureHandle(ql.FlatForward(calculation_date, r, ql.Actual360DayCounter)),
            ql.YieldTermStructureHandle(ql.FlatForward(calculation_date, 0, ql.Actual360DayCounter)),
            ql.BlackVolTermStructureHandle(ql.BlackConstantVol(calculation_date, ql.Actual360DayCounter, 0.2))
        )
        option.setPricingEngine(ql.AnalyticEuropeanEngine(bs_process))
        try:
            implied_vol = option.impliedVolatility(model_price, bs_process, 1e-6, 1)
            implied_vol_surface[i][j] = implied_vol
        except RuntimeError:
            implied_vol_surface[i][j] = np.nan

# Afficher la surface de volatilité implicite
X, Y = np.meshgrid(strike_grid, maturity_grid)
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d') # Utilisez add_subplot avec projection=3d
surf = ax.plot_surface(X, Y, implied_vol_surface, cmap=cm.viridis)
ax.set_xlabel('Prix d'exercice')
ax.set_ylabel('Maturité (années)')
ax.set_zlabel('Volatilité Implicite')
plt.title('Surface de Volatilité Implicite du Modèle de Heston')
plt.show()

```

Surface de Volatilité Implicite du Modèle de Heston



On constate que le smile de volatilité est toujours présent avec des volatilités implicites plus élevées lorsqu'elles ne sont pas à la monnaie. De plus, la présence d'un skew négatif de volatilité dans la surface indique que les agents s'attendent à une baisse du prix de l'actif sous-jacent dans les échéances à venir et augmentent relativement la demande de call Out of the money afin de se couvrir du risque potentiel. Cela peut aussi représenter l'asymétrie des rendements de l'actif sous-jacent, le marché va anticiper une baisse de prix plus impactante que la hausse, la volatilité implicite des calls OTM va donc s'élever encore plus. Une stratégie de spread vertical pourrait être employée par un gestionnaire de risque afin de tirer profit de cette situation.

Conclusion

A travers l'expérimentation de ce projet, nous avons appris de la différence fondamentale qui résulte de ses deux modèles. La volatilité stochastique et sa calibration possèdent une bien meilleure représentation du marché qu'une volatilité qui demeure constante. Différentes stratégies vont s'établir autour de cet actif qui risque de voir potentiellement son prix diminuer dans les semaines à venir même si cela ne représente pas non plus une baisse significative certaine. La détermination de la demande d'option et des paramètres liés à ces

anticipations semblent assez importante dans le pricing d'une option car à partir de la volatilité implicite, on peut se projeter dans l'avenir et se préparer face à différents chocs affectant l'actif sous-jacent.